

Smart Sports Equipment Management System

Jaspreet Singh Sanveer Singh Harshjot Singh

August 2026

1 Introduction

The Smart Sports Equipment Management System (SSEMS) is a web-based platform designed to digitize the process of issuing and returning sports equipment at a college sports complex. Traditionally, this process requires students to physically deposit their identity card in exchange for equipment, with records maintained manually in a register. SSEMS replaces this workflow with a QR-code-based digital system that tracks inventory in real time and ensures equitable access to high-demand equipment through a fairness-aware scheduling algorithm.

2 Motivation

The manual issue-return process at college sports complex suffers from several shortcomings:

- Lack of a centralized, searchable record of equipment availability and condition.
- No visibility into overdue or damaged items until physically checked.
- High-demand equipment (e.g courts, basketballs) is frequently monopolized by the same group of students, leaving others with little or no access.
- Manual record-keeping is time-consuming for sports complex staff and error-prone.

A digital system addresses these inefficiencies while introducing accountability and fairness into equipment allocation.

3 Problem Statement

Given a sports complex with a finite inventory of equipment and limited time-slots for high-demand resources (courts, popular equipment), design and implement a system that:

1. Digitizes student identification and equipment issue/return, eliminating the need for physical ID card deposits.
2. Maintains a real-time, centralized inventory record accessible to stakeholders (staff/administration).
3. Implements a scheduling mechanism that distributes access to high-demand equipment fairly, preventing monopolization by a subset of students.

4 Proposed Solution

SSEMS is composed of the following core modules:

- **Authentication & Digital ID:** Each student is issued a unique QR code (derived from their student ID) used for identification instead of a physical card.
- **Inventory Management:** All equipment items are tracked with real-time status (available, issued, under maintenance, damaged).
- **Issue/Return Workflow:** Staff scan a student's QR code and the corresponding item to create a transaction record, with automatic due-time assignment.
- **Fairness Scheduling Algorithm:** For high-demand items, a priority-weighted queue and hard usage caps ensure equitable distribution of access (detailed in Section 8).
- **Admin Dashboard:** Displays live stock levels, defaulter lists, and usage analytics.
- **Notifications:** Automated reminders for return deadlines.

5 System Architecture and Tech Stack

- **Backend:** FastAPI (Python), providing REST APIs for all core operations.
- **ORM & Database:** SQLAlchemy with PostgreSQL as the relational data store.
- **Frontend:** React (with Streamlit used for early-stage MVP prototyping).
- **QR Code Handling:** The `qrcode` Python library for generation and `html5-qrcode` (browser-based) for scanning.
- **Containerization & Deployment:** Docker Compose to orchestrate backend, database, and frontend services; deployed on a free-tier platform such as Render or Railway.
- **Notifications:** SMTP-based email reminders for due/overdue equipment.

6 System Diagrams

6.1 Data Flow Diagram — Level 0

The Level 0 Data Flow Diagram shows the interaction of the complete system with the external entities: Student, Staff, Admin, and Notification Service.

Data Flow Diagram for Smart Sports Equipment Management System

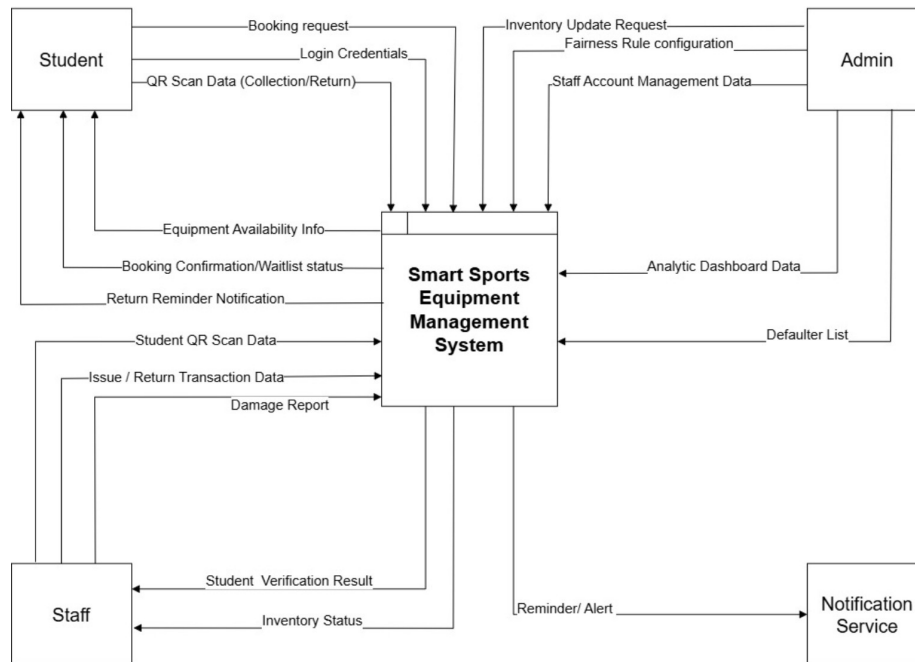


Figure 1: Data Flow Diagram (Level 0 / Context Diagram)

6.2 Data Flow Diagram — Level 1

The Level 1 Data Flow Diagram decomposes the system into its major internal processes and data stores, including registration and login, booking and scheduling, fairness computation, issue/return transactions, inventory management, and notifications and analytics.

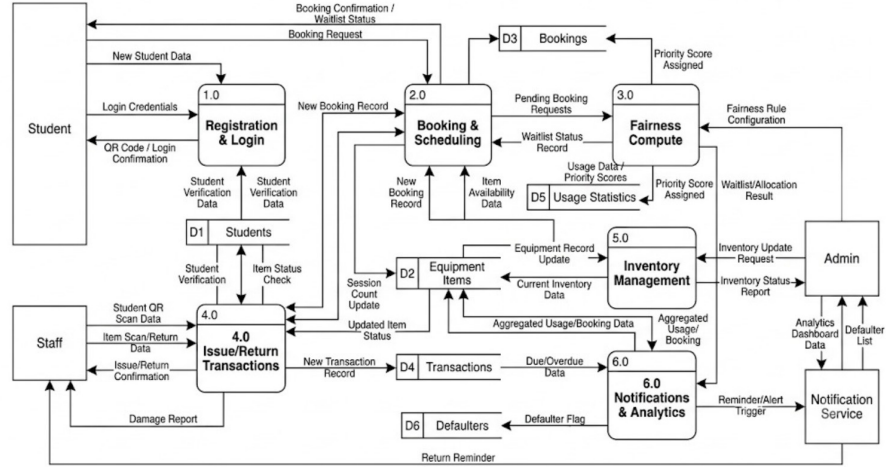


Figure 2: Data Flow Diagram (Level 1)

6.3 Entity Relationship Diagram

The Entity Relationship Diagram represents the principal database entities and their relationships, including Students, Equipment Items, Bookings, Transactions, Usage Statistics, and Defaulters.

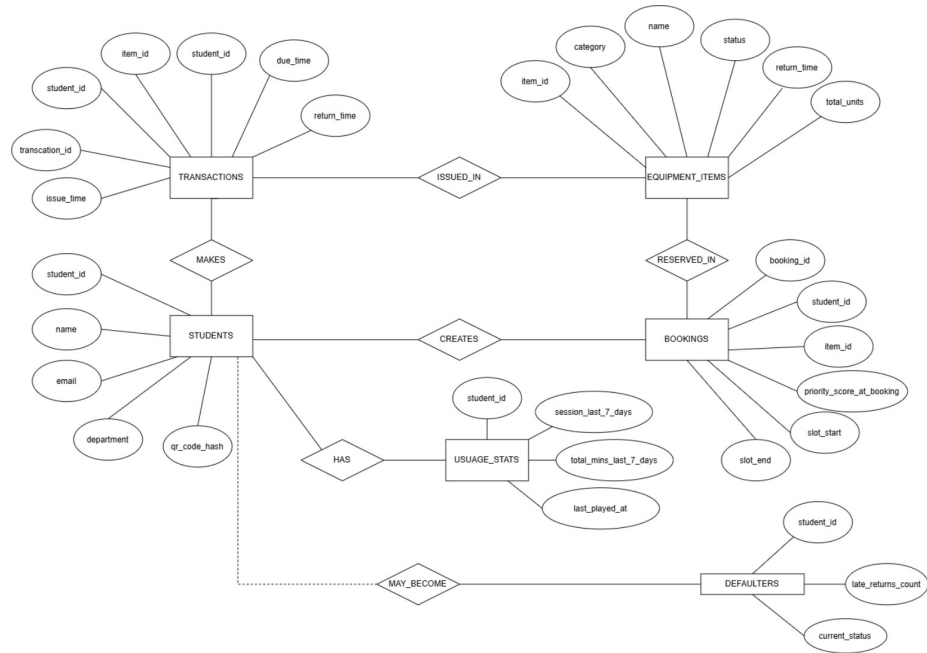


Figure 3: Entity Relationship Diagram

6.4 Use Case Diagram

The Use Case Diagram presents the major system functionalities from the perspective of Student, Staff, Admin, and the System Notification Service.

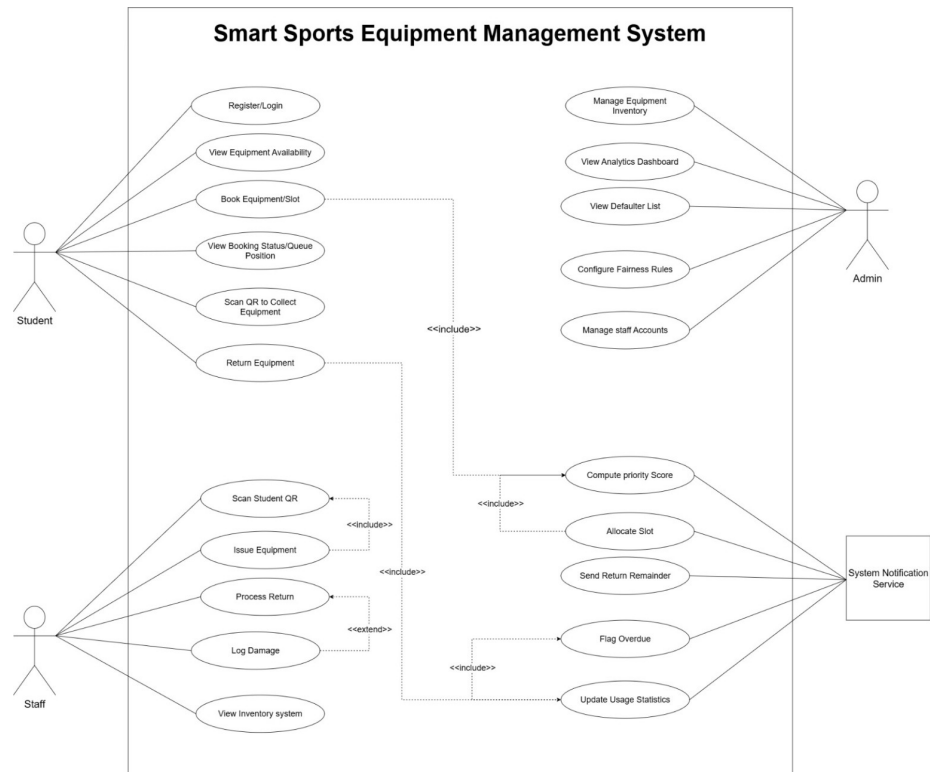


Figure 4: Use Case Diagram

7 Database Schema

The relational schema consists of six primary entities.

7.1 Students

- `student_id` (Primary Key)
- `name`, `email`, `department`
- `qr_code_hash` — unique token embedded in the student's QR code

7.2 Equipment Items

- `item_id` (Primary Key)
- `category`, `name`
- `status` — `available` / `issued` / `maintenance` / `damaged`
- `total_units` — for non-unique, bulk-tracked items

7.3 Transactions

- `transaction_id` (Primary Key)
- `student_id`, `item_id` (Foreign Keys)
- `issue_time`, `due_time`, `return_time`
- `condition_on_return`, `late_flag`

7.4 Bookings

Used for slot-based equipment such as courts.

- `booking_id` (Primary Key)
- `student_id`, `item_id` (Foreign Keys)
- `slot_start`, `slot_end`
- `status` — `confirmed` / `waitlisted` / `cancelled` / `completed` / `no_show`
- `priority_score_at_booking` — snapshot for auditability

7.5 Usage Statistics

A denormalized table updated on each transaction/booking closure, avoiding recomputation from the full transaction history.

- `student_id` (Primary Key, Foreign Key)
- `sessions_last_7_days`, `total_minutes_last_7_days`
- `last_played_at`

7.6 Defaulters

- `student_id`
- `late_returns_count`, `current_status`

8 Fairness Scheduling Algorithm

When demand for a slot or item exceeds available supply, waiting students are ranked using a *priority score*, where a lower score indicates higher priority.

8.1 Priority Score Formula

$$S_{norm} = \frac{sessions_last_7_days}{\max(sessions_last_7_days)_{allstudents}} \quad (1)$$

$$P_{recency} = \{ (2 - d) \times 0.5, if d < 20, if d \geq 20 \} \quad (2)$$

where d is the number of days since the student last played.

$$priority_score = S_{norm} + P_{recency} \quad (3)$$

8.2 Allocation Procedure

1. Collect all booking requests for a given slot/item.
2. Sort requests in ascending order of `priority_score`.
3. Allocate the slot(s) to the lowest-scoring student(s) up to capacity.
4. Remaining students are placed on a waitlist, carried forward to the next available slot.
5. A hard cap is enforced independently of the score: if `sessions_last_7_days` $\geq$ `daily_cap`, the student is blocked from further bookings that day.
6. Ties are broken using first-come-first-served (request timestamp).

This two-layer design — a soft priority ranking combined with a hard usage cap — ensures fairness while remaining explainable and simple to justify.

9 Implementation Methodology

The system is developing in the following phases:

1. **Environment Setup:** Project repository initialized with separate back-end/frontend directories; Python virtual environment configured with FastAPI, SQLAlchemy, and Uvicorn; PostgreSQL provisioned via Docker.
2. **Database Schema Implementation:** Entities translated into SQLAlchemy classes; Alembic used for migration management; database seeded with sample data for testing.
3. **Core API Development:** CRUD endpoints implemented for equipment registration, student registration, and issue/return operations, validated using Postman/Thunder Client.
4. **QR-Based Identification:** Per-student QR codes generated using the `qrcode` library; a scanning interface built using `html5-qrcode` for staff to identify students without manual entry.
5. **Fairness Scheduling Logic:** Usage statistics updated on transaction closure; the priority score computed as an independently testable module; integrated into the booking endpoint with cap enforcement.
6. **Frontend Development:** A React-based interface built with separate student (browse/book equipment, view queue position) and staff/admin (scan, dashboard, defaulter list) views.
7. **Additional Features:** Email notifications for due returns, analytics dashboards (peak hours, most-borrowed items), and damage logging on return.
8. **Containerization and Deployment:** Backend and frontend containerized with Docker; orchestrated via Docker Compose; deployed to a cloud platform for live demonstration.

10 Conclusion and Future Scope

SSEMS demonstrates how a manual, paper-based process can be digitized to improve accountability, transparency, and fairness in resource allocation. Potential extensions include:

- A machine learning model to predict no-show probability for bookings, enabling smart overbooking.
- An app deployed in real time used by Thapar students in real time.
- Gamification (leaderboards) to encourage timely returns.
- Integration with the college's existing SSO/authentication infrastructure.